

第3章_运算方法与运算器_复习重点

第3章 运算方法与运算器 - 复习重点

依据《期末复习要点2026.pdf》中“数的表示以及运算”部分抽取第 3 章相关重点。

PDF 中与本章直接对应的复习重点：定点运算器溢出判断电路、补码乘法的基本原理、浮点数的规格化。

每个重点包含：原知识点、原知识点对应例题、OCR题目与答案。

重点1：定点运算器溢出判断电路

期末 PDF 对应重点：定点运算器溢出判断电路。

原知识点与例题

3.2.2 溢出及检测

1. 溢出的概念

溢出：两个数相加（相减）的结果超出了补码的表示范围。

【例3.5】 溢出示例

(1) 设 $[x]_{\text{补}} = 0.1011$, $[y]_{\text{补}} = 0.1100$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 0.1011 \\ +0.1100 \\ \hline 1.0111 \end{array} \quad (\text{两个正数相加结果为负数, 出错!})$$

原因: $x + y = 11/16 + 12/16 = 23/16 > 1$, 超出定点小数表示范围 $[-1, 1)$

(2) 设 $[x]_{\text{补}} = 1.0101$, $[y]_{\text{补}} = 1.0100$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 1.0101 \\ +1.0100 \\ \hline 10.1001 \end{array} \quad (\text{两个负数相加结果为正数, 出错!})$$

原因: $x + y = (-11/16) + (-12/16) = -23/16 < -1$, 超出表示范围

2. 溢出检测

三种常见的溢出检测方法:

方法一: 根据操作数和运算结果的符号位是否一致

运算情况	是否溢出
正数 + 正数 = 正数	不溢出
正数 + 正数 = 负数	溢出
负数 + 负数 = 负数	不溢出
负数 + 负数 = 正数	溢出
正数 + 负数	不可能溢出

设 X_f 、 Y_f 为操作数的符号位， S_f 为运算结果的符号位：

$$V = \overline{X_f} \cdot \overline{Y_f} \cdot S_f + X_f \cdot Y_f \cdot \overline{S_f}$$

- $V = 0$ ：没有溢出
- $V = 1$ ：有溢出

【例3.6】 $x = -0.1011, y = 0.1100$, 求 $[x - y]_{\text{补}}$

解： $[x]_{\text{补}} = 1.0101, [y]_{\text{补}} = 0.1100, [-y]_{\text{补}} = 1.0100$

负数 - 正数 = 负数 + 负数，结果为正数，溢出了。

$$\begin{array}{r} 1.0101 \\ +1.0100 \\ \hline 10.1001 \end{array}$$

$X_f = 1, Y_f = 1$ ($[-y]_{\text{补}}$ 的符号位), $S_f = 0$

$$V = 1 \cdot 1 \cdot 1 + 0 \cdot 0 \cdot 0 = 1 \quad (\text{溢出})$$

方法二：根据最高数据位的进位与符号位的进位是否一致

$$V = C_f \oplus C_d$$

- C_f ：符号位产生的进位信号
- C_d ：最高有效位产生的进位信号

【例3.7】 $x = -111, y = 110$, 求 $[x - y]_{\text{补}}$

解： $[x]_{\text{补}} = 1,001, [y]_{\text{补}} = 0,110, [-y]_{\text{补}} = 1,010$

$$\begin{array}{r} 1,001 \\ +1,010 \\ \hline 10,011 \end{array}$$

$C_f = 1, C_d = 0, V = 1 \oplus 0 = 1$ (溢出, $-7 - 6 = -13$ 超出 $-8 \sim +7$ 范围)

【例3.8】 $x = -011, y = 100$, 求 $[x - y]_{\text{补}}$

解： $[x]_{\text{补}} = 1,101, [y]_{\text{补}} = 0,100, [-y]_{\text{补}} = 1,100$

$$\begin{array}{r} 1,101 \\ +1,100 \\ \hline 11,001 \end{array}$$

$C_f = 1, C_d = 1, V = 1 \oplus 1 = 0$ (没有溢出, $-3 - 4 = -7$ 在范围内)

方法三: 利用变形补码的符号位进行检测

变形补码 (双符号位补码): 正数的符号位为00, 负数的符号位为11。

- 运算结果符号位为 00 或 11: 没有溢出
- 运算结果符号位为 01 或 10: 产生溢出

$$V = S_{f1} \oplus S_{f2}$$

- S_{f1} 、 S_{f2} 分别表示运算结果的第1个符号位和第2个符号位
- S_{f1} 为结果真正符号位

【例3.9】 变形补码溢出检测

(1) 设 $[x]_{\text{补}} = 00.1011, [y]_{\text{补}} = 00.0111$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 00.1011 \\ +00.0111 \\ \hline 01.0010 \quad (\text{符号位}01, \text{正溢出}) \end{array}$$

$$x + y = 11/16 + 7/16 = 18/16 > 1$$

(2) 设 $[x]_{\text{补}} = 11.0101, [y]_{\text{补}} = 11.0011$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 11.0101 \\ +11.0011 \\ \hline 10.1000 \quad (\text{符号位}10, \text{负溢出}) \end{array}$$

$$x + y = -11/16 - 13/16 = -24/16 < -1$$

(3) 设 $[x]_{\text{补}} = 00.1011, [y]_{\text{补}} = 11.0011$, 求 $[x + y]_{\text{补}}$

$$\begin{array}{r} 00.1011 \\ +11.0011 \\ \hline 11.1110 \quad (\text{符号位}11, \text{没有溢出}) \end{array}$$

$$x + y = 11/16 - 13/16 = -2/16$$

图片信息: PPT第21页, 溢出检测软件方法示意图

3.2.3 加减法的逻辑实现

1. 全加器 (Full Adder, FA)

全加器真值表:

X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

逻辑表达式：

$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

图片信息：PPT第23页，全加器逻辑电路实现（假设异或门延迟3T，与门和或门延迟1T）

2. 半加器 (Half Adder, HA)

$$S_i = X_i \oplus Y_i$$

$$C_{i+1} = X_i \cdot Y_i$$

图片信息：PPT第27页，半加器真值表和电路图

3. 多位串行加法器（行波进位加法器）

由 n 个全加器构成 n 位串行加法器。

- 无符号加法溢出标志： $V = C_n$
- 有符号加法溢出标志： $V = C_n \oplus C_{n-1}$

图片信息：PPT第28页，图3.3 多位串行加法器

4. 可控加减法电路 (Controller Adder/Subtractor)

- $Sub = 0$ ：加法运算
- $Sub = 1$ ：减法运算 ($X - Y = X + /Y + 1$)

原理： $Y_i \oplus Sub = 1$ 相当于取反（求反码），结果 $+1$ 体现在 $C_0 = Sub = 1$ 。

- 无符号加法溢出： $V = C_n$
- 无符号减法溢出： $V = /C_n$
- 有符号加法/减法溢出： $V = C_n \oplus C_{n-1}$

图片信息：PPT第30页，图3.4 多位可控加减法电路

图片信息：PPT第31页，可控加减法溢出判断示意图

5. 先行进位加法器 (Carry Look-Ahead Adder, CLA)

进位生成函数： $G_i = X_i Y_i$

进位传递函数： $P_i = X_i \oplus Y_i$

$$C_{i+1} = G_i + P_i C_i$$

展开得：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

4位先行进位电路：时间延迟为2T（1T与门 + 1T或门）

图片信息：PPT第34页，图3.6 4位进位生成/进位传递电路

图片信息：PPT第35页，图3.5 4位先行进位电路

成组进位函数：

$$G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$$

$$P^* = P_3 P_2 P_1 P_0$$

$$C_4 = G^* + P^* C_0$$

图片信息：PPT第37页，图3.5 可级联的4位先行进位电路（CLA 74182）

4位先行进位加法器（快速加法器）：

- 得到 S_0 的时间延迟：6T（ P_0 生成3T + 异或门3T）
- 得到 S_1 、 S_2 、 S_3 的时间延迟：8T（再加2T进位电路）
- 得到 C_4 的时间延迟：5T

图片信息：PPT第38页，图3.6 4位先行进位加法器

16位加法器实现方式：

方式	结构	时间延迟
串行（图3.3）	—	$(2n + 4)T = 36T$
组内并行、组间串行（图3.7）	4个4位CLA串联	14T
组内并行、组间并行（图3.8）	4个4位CLA + 可级联先行进位电路	12T

图片信息：PPT第41页，图3.7 16位加法器（组内并行、组间串行）

图片信息：PPT第42页，图3.8 16位加法器（组内并行、组间并行）

OCR题目与答案

补码加减与溢出检测

选择题4 [2018]：若 int 型变量 x 和 y 的机器数分别为 FFFF FFDFH 和 0000 0041H，则 x、y 及 x-y 的机器数为__。

A. x=-65, y=41, x-y 溢出 B. x=-33, y=65, x-y=FFFF FF9DH C. x=-33, y=65, x-y=FFFF FF9EH D. x=-65, y=41, x-y=FFFF FF96H

答案：C。解析：FFFF FFDFH 对应 -33，0000 0041H 对应 65，按补码减法得 FFFF FF9EH。

综合3.4：用变形补码计算 x+y 并判断溢出。

(1) x=0.11010, y=0.101110: $[x+y]_{\text{补}} = 01.10001$ ，正溢出。

(2) x=0.11101, y=-0.10100: $[x+y]_{\text{补}} = 00.01001$ ，未溢出。

(3) x=-0.10111, y=-0.11000: $[x+y]_{\text{补}} = 10.10001$ ，负溢出。

综合3.5：用变形补码计算 x-y 并判断溢出。

(1) x=0.11011, y=0.11101: $[x-y]_{\text{补}} = 11.11110$ ，未溢出。

(2) x=0.10111, y=0.11110: $[x-y]_{\text{补}} = 11.11001$ ，未溢出。

(3) x=-0.11111, y=-0.11001: $[x-y]_{\text{补}} = 11.11010$ ，未溢出。

加法器、可控加减与溢出标志

简答1：为什么采用并行进位能提高加法器的运算速度？

答案：并行进位电路提前产生各位所需进位信号，各位求和不再依赖低位运算结果，可并行进行。

综合3.11：8 位机器中 unsigned int x=134; unsigned int y=246; int m=x; int n=y; unsigned int z1=x-y; unsigned int z2=x+y; int k1=m-n; int k2=m+n;。

答案要点：R1= 86H，R5= 90H，R6= 7CH；m=-122，k1=-112；无符号加减和补码有符号加减可复用同一个 n 位加法器及辅助电路；若符号位进位 C_f 与最高数据位进位 C_d 不同则溢出，k2=m+n 发生溢出。

综合3.12：比较图 3.2(b) 实现全加器时图 3.3~图 3.8 电路的延迟和硬件成本。

答案要点：(b) 方案增加少量或门成本，但串行加法器、串行可控加减法电路关键延迟可减少约 $3T$ ；4 位快速加法器延迟为 $6T$ 、进位延迟为 $3T$ ；16 位组内并行/组间串行和组内并行/组间并行加法器关键延迟均可比 (a) 方案少约 $2T$ ，成本增加 16 个或门。可按速度与成本需求选择。

重点2：补码乘法的基本原理

期末 PDF 对应重点：补码乘法的基本原理（教材对应流程图、逻辑框图、实验中对应的 8 位补码一位乘法电路工作原理）。

原知识点与例题

3.3.2 补码一位乘法（Booth算法）

1. Booth算法公式推导

(1) 被乘数 x 符号任意，乘数 y 符号为正数：

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot y$$

(2) 被乘数 x 符号任意，乘数 y 符号为负数：

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot 0.y_1y_2 \dots y_n - [x]_{\text{补}}$$

(3) 被乘数 x 符号任意，乘数 y 符号任意（Booth算法）：

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot 0.y_1y_2 \dots y_n - [x]_{\text{补}} \cdot y_0$$

图片信息：PPT第52页，图3.11 补码一位乘法算法流程图

2. 补码一位乘法运算

【例3.11】 $[x]_{\text{补}} = 1.0111, [y]_{\text{补}} = 1.0011$ ，用补码一位乘法求 $[x \cdot y]_{\text{补}}$

解： $[-x]_{\text{补}} = 0.1001$ （取反加1）

部分积	乘数 y	y_{n+1}	说明
00.0000	10011	0	$P = 0$
+ 00.1001			$y_n y_{n+1} = 10, +[-x]_{\text{补}}$
00.1001	10011		
→ 00.0100	11001	1	算术右移一位
+ 00.0000			$y_n y_{n+1} = 11, +0$
00.0100	11001		
→ 00.0010	01100	1	算术右移一位
+ 11.0111			$y_n y_{n+1} = 01, +[x]_{\text{补}}$
11.1001	01100		
→ 11.1100	10110	0	算术右移一位
+ 00.0000			$y_n y_{n+1} = 00, +0$
11.1100	10110		

部分积	乘数 y	y_{n+1}	说明
$\rightarrow 11.1110$	01011	0	算术右移一位
+ 00.1001			$y_n y_{n+1} = 10$, $+[-x]_{\text{补}}$
00.0111	0101		最后一步不移位

$$[x \cdot y]_{\text{补}} = 0.0111\ 0101$$

验证: $x = -0.1001, y = -0.1101, x \cdot y = 0.0111\ 0101$

3. 补码一位乘法的逻辑实现

图片信息: PPT第54页, 图3.12 补码一位乘法逻辑实现

OCR题目与答案

补码乘法与Booth算法

选择题2 [2010]: 8 位补码 $r_1 = FEH$, $r_2 = F2H$, $r_3 = 90H$, $r_4 = F8H$, 哪项乘法结果存入 8 位寄存器会溢出?

A. $r_1 \times r_2$ B. $r_2 \times r_3$ C. $r_1 \times r_4$ D. $r_2 \times r_4$

答案: B。解析: 真值分别为 -2、-14、-112、-8, $r_2 \times r_3 = 1568$ 超出 8 位补码范围。

综合3.7: 用补码一位乘法计算 $x \times y$ 。

(1) $x=0.10110$, $y=-0.00011$, $[x \times y]_{\text{补}} = 1.1110111110$ 。

(2) $x=-0.011010$, $y=-0.011101$, $[x \times y]_{\text{补}} = 0.001011110010$ 。

解析要点: 建议符号位采用双符号, 右移规则为算术右移。

重点3: 浮点数的规格化

期末 PDF 对应重点: 浮点数的规格化。

原知识点与例题

3.5.1 浮点加减法运算

1. 阶码和尾数采用补码表示的浮点加减运算

运算步骤:

步骤	操作	说明
(1) 对阶	小阶向大阶对齐	阶码小的尾数右移, 阶码加1
(2) 尾数运算	定点数的补码加减运算	

步骤	操作	说明
(3) 结果规格化	右规或左规	尾数绝对值>1则右规（右移1次，阶码+1）；尾数绝对值<0.5则左规（左移若干次，阶码-1）
(4) 舍入	末位恒置1法或0舍1入法	右规时尾数末尾被丢弃产生误差
(5) 溢出判断	阶码符号位为01或10时溢出	

【例3.14】 设 $x = 2^{-101} \cdot (-0.101011)$, $y = 2^{-010} \cdot (0.001110)$, 阶码3位, 尾数6位 (均不含符号位), 都用补码表示, 计算 $x + y$

解:

$$[x]_{\text{补}} = 11\ 011, 11.010101 \quad [y]_{\text{补}} = 11\ 110, 00.001110$$

(1) **对阶**: x (阶码=-5) 向 y (阶码=-2) 对齐, x 的尾数右移3位

对阶后 $[x]_{\text{补}} = 11\ 110, 11.111010(101)$

(2) **尾数运算**:

$$\begin{array}{r} 11.111010(101) \\ + 00.001110 \\ \hline 00.001000(101) \end{array}$$

(3) **规格化**: 尾数 00.001000(101) 为非规格化数 (绝对值太小), 左规2次

规格化后尾数 = 00.100010(1), 阶码减2变为 11 100

(4) **舍入**: 0舍1入法, 尾数 = 00.100011

(5) **溢出判断**: 阶码双符号位相同 (11), 没有溢出

$$[x + y]_{\text{补}} = 11\ 100, 00.100011, \quad x + y = 2^{-100} \cdot (0.100011)$$

【例3.15】 设 $x = 2^7 \cdot (25/32)$, $y = 2^6 \cdot (-23/32)$, 阶码5位, 尾数7位 (含2位符号位), 计算 $x - y$ (0舍1入法)

解:

$$[x]_{\text{补}} = 00\ 111, 00.11001 \quad [-y]_{\text{补}} = 00\ 110, 00.10111$$

(1) **对阶**: y (阶码=6) 向 x (阶码=7) 对齐, y 的尾数右移1位

对阶后 $[-y]_{\text{补}} = 00\ 111, 00.01011(1)$

(2) **尾数运算**:

$$\begin{array}{r}
 00.11001 \\
 +00.01011(1) \\
 \hline
 01.00100(1)
 \end{array}$$

(3) **规格化**：尾数 01.00100(1) 绝对值太大，右规1次

规格化后尾数 = 00.10010(01)，阶码加1变为 01 000

(4) **舍入**：0舍1入法，尾数 = 00.10010

(5) **溢出判断**：阶码双符号位为01，发生溢出

$$[x - y]_{\text{补}} = 01\ 000, 00.10010 \quad (\text{溢出})$$

验证： $x = 100, y = -46, x - y = 146$ （超出 $-128 \sim +127$ 范围）

2. IEEE754浮点数的加减运算

IEEE754浮点数特点：阶码用移码表示，尾数用原码表示（尾数 = $1.M$ ，最高位隐藏）。

结果规格化：

- 尾数 = $1.xx \dots x$ ：规格化尾数
- 尾数 = $1x.xx \dots x$ ：绝对值 >2 ，需1次右规（尾数右移1次，阶码+1）
- 尾数 = $0.xx \dots x$ ：绝对值 <1 ，需左规（左移若干次直到 $1.xx \dots x$ ，每左移1次阶码-1）

舍入处理：就近舍入、朝 $+\infty$ 方向舍入、朝 $-\infty$ 方向舍入、朝0方向舍入

溢出判断：

- 右规时阶码为全1（移码全1为最大正数）：规格化**上溢**
- 左规时阶码为全0（移码全0为最小负数）：规格化**下溢**

【例3.16】 两个IEEE754单精度浮点数 $X = 00C0\ 0000H, Y = 0080\ 0000H$ ，求 $X - Y$

解：

$$X = 0\ 00000001\ 1000 \dots 0 = 1.1 \times 2^{-126}$$

$$Y = 0\ 00000001\ 0000 \dots 0 = 1.0 \times 2^{-126}$$

(1) **对阶**：阶码相同，无需对阶

(2) **尾数相减**： $1.1 - 1.0 = 0.1$

(3) **规格化**：0.1 为非规格化数，左移1次，尾数变为 1.0，阶码减1变为 00000000（全0）

(4) **舍入**：无需舍入

(5) **溢出判断**：阶码全0，规格化下溢

IEEE754非规格化浮点数： $(-1)^S \times 0.M \times 2^{-126}$

$$X - Y = 0.1 \times 2^{-126} = 2^{-127} = 0040\ 0000H$$

OCR题目与答案

浮点加减、规格化与溢出

选择题6 [2019]: 阶码 5 位、尾数 7 位且均为补码, $X = 2^7 \times 29/32$, $Y = 2^5 \times 5/8$, 求 $X+Y$ 。

答案: D, 发生溢出。**解析:** 对阶、尾数相加后结果上溢, 右规使阶码变为 01000, 阶码符号位 01, 判定溢出。

选择题7 [2015]: 浮点加减运算中, I 对阶不引起阶码溢出; II 右规和舍入可能引起上溢; III 左规可能引起下溢; IV 尾数溢出时结果不一定溢出。

答案: D。**解析:** 四项均正确。

简答2: 如何判断浮点数运算结果是否发生溢出?

答案: 看阶码是否溢出。采用双符号位阶码时, 双符号位不同表示浮点数溢出。

简答3: 如何判断并处理浮点运算结果规格化?

答案: 双符号位尾数若不是 11.0... 或 00.1... 形式则非规格化。尾数符号位为 01 或 10 时右规并阶码加 1; 尾数为 11.1... 或 00.0... 时左规, 尾数左移多少位阶码减多少。

综合3.9: 机器补码浮点加法。

(1) $x = 2^{011} \times 0.100100$, $y = 2^{010} \times (-0.011010)$, 结果 $[x+y]_{\text{补}} = 00010, 00101110$, 无溢出。

(2) $x = 2^{-101} \times (-0.100010)$, $y = 2^{-100} \times (-0.010110)$, 结果 $[x+y]_{\text{补}} = 11100, 11011001$, 无溢出。

综合3.10: IEEE754 单精度浮点加减。

(1) $0.625 + (-12.25)$ 的结果为 C13A0000H。

(2) $0.625 - (-12.25)$ 的结果为 414E0000H。

解析要点: 先对阶, 再做尾数加减; 两个结果尾数均符合 1.X 形式, 无须规格化, 阶码正常, 无溢出。

复习重点速查表

PDF复习重点	对应知识点	对应原例题	对应OCR题目
定点运算器溢出判断电路	3.2.2 溢出及检测; 3.2.3 加减法的逻辑实现	例3.5-例3.9; 全加器、可控加减、先行进位加法器相关例示	选择4; 综合3.4; 综合3.5; 综合3.11; 综合3.12; 简答1
补码乘法的基本原理	3.3.2 补码一位乘法 (Booth算法)	例3.11	选择2; 综合3.7

PDF复习重点	对应知识点	对应原例题	对应OCR题目
浮点数的规格化	3.5.1 浮点加减法运算	例3.14；例3.15；例3.16	选择6；选择7；简答2；简答3；综合3.9；综合3.10